

5. When Indexes Don't Work

Audience: Backend engineers, and chapter team members working with PostgreSQL in production.

Talk goal: Explain why PostgreSQL sometimes ignores an index, how to prove the reason, and which fixes are actually worth applying in real systems.

Core message: An index is not a promise. PostgreSQL chooses the cheapest execution plan based on statistics, selectivity, and cost estimates.

1. Executive summary

This document turns the classic “why is PostgreSQL not using my index?” question into a production-minded guide. The central split is simple: either PostgreSQL cannot use the index, or it can use it but estimates that another plan will be cheaper.

The correct fix is not always “add another index”. In many cases, the best answer is changing the query shape, fixing pagination, refreshing statistics, or adjusting the index definition so that it matches the real access pattern.

2. How to investigate index behavior

1. Start with EXPLAIN, and then use EXPLAIN ANALYZE when you want actual timing and row counts.
2. Compare estimated rows vs actual rows. Large mismatches usually point to stale or poor statistics.
3. Check whether the plan is Seq Scan, Index Scan, Bitmap Heap Scan, or Index Only Scan.
4. Temporarily discourage sequential scans in a non-production session with SET enable_seqscan = off to learn whether PostgreSQL can use the index at all.
5. If needed, hide competing indexes in a test transaction to see whether the planner prefers a different index.

3. Representative EXPLAIN patterns

The examples below are intentionally written as sample plans so you can reuse them as live demos.

3.1 Low selectivity: the index exists, but Seq Scan is cheaper

Scenario: a million users, 900k of them have the same country value.

```
CREATE INDEX idx_users_country ON users(country);

EXPLAIN ANALYZE
SELECT *
FROM users
WHERE country = 'Azerbaijan';

Seq Scan on users  (cost=0.00..18334.00 rows=900000 width=16)
(actual time=0.021..112.943 rows=900000 loops=1)
  Filter: (country = 'Azerbaijan')
  Rows Removed by Filter: 100000
Planning Time: 0.213 ms
Execution Time: 131.550 ms
```

Why this happens: when a predicate returns a very large fraction of the table, random heap visits through the index can be more expensive than reading the table sequentially.

3.2 Highly selective predicate: Index Scan becomes attractive

Same table, but a rare value.

```
EXPLAIN ANALYZE
SELECT *
FROM users
WHERE country = 'Georgia';
```

```
Index Scan using idx_users_country on users (cost=0.42..1820.44 rows=50000 width=16)
(actual time=0.041..12.192 rows=50000 loops=1)
Index Cond: (country = 'Georgia')
Planning Time: 0.153 ms
Execution Time: 13.001 ms
```

STATISTICS is actually **the most important topic** behind "When Indexes Don't Work".

Many engineers think:

PostgreSQL checks whether an index exists, then decides whether to use it.

That's not how it works.

PostgreSQL does:

1. Parse query
2. Generate possible execution plans
3. Estimate cost of each plan
4. Choose cheapest plan

The critical question is: How does PostgreSQL estimate the cost?

Answer:

It uses STATISTICS.

What are PostgreSQL Statistics?

Statistics are metadata about table data that PostgreSQL collects during:

`ANALYZE users;` or automatically via: Autovacuum

Statistics are stored in system catalogs.

Example: `SELECT * FROM pg_stats WHERE tablename = 'users';`

Example Table

```
CREATE TABLE users (
  id BIGSERIAL,
  country TEXT
);
```

Data:

country
Azerbaijan

Azerbaijan
Azerbaijan
Azerbaijan
Turkey
Georgia

Suppose: 1,000,000 rows

Distribution:

Azerbaijan = 900,000
 Turkey = 50,000
 Georgia = 50,000

After: ANALYZE users;

PostgreSQL stores something similar to:

country

Most Common Values:

 Azerbaijan

Frequency:

 90%

Why Does PostgreSQL Need Statistics?

Suppose query: `SELECT * FROM users WHERE country = 'Azerbaijan';`

Planner must answer: How many rows will this return?

Without statistics: No idea

With statistics: About 900,000 rows

This estimate directly affects: Seq Scan vs Index Scan

Real Planner Decision

Imagine: `CREATE INDEX idx_country ON users(country);`

Query: `SELECT * FROM users WHERE country='Azerbaijan';`

Statistics say: 90% of table

Planner estimates: 900,000 rows

Cost calculation:

Option A

Index Scan

Needs: 900,000 index entries + 900,000 heap fetches

Expensive.

Option B

Seq Scan

Needs: Read table once

Cheaper.

Planner chooses: Seq Scan

Same Index, Different Value

Query: `SELECT * FROM users WHERE country='Georgia';`

Statistics: 5%

Planner estimates: 50,000 rows

Now: Index Scan is cheaper.

Same index. Same table. Different statistics. Different plan.

Why Statistics Become Dangerous

Suppose:

Yesterday: Azerbaijan = 90%

Planner learned: 90%

Then application inserts:

500,000 new Georgia users

But ANALYZE hasn't run yet.

Actual distribution:

Azerbaijan = 60%
Georgia = 35%
Turkey = 5%

Planner still believes: Azerbaijan = 90%

because statistics are stale.

Now query: `SELECT * FROM users WHERE country='Georgia';`

Planner thinks: 50,000 rows

Actual: 350,000 rows

Bad estimate. Bad plan. Slow query.

How To Detect Bad Statistics

Most important debugging skill:

Look at: EXPLAIN ANALYZE

Example:

Index Scan Estimated Rows: 100

Actual Rows: 500000

Huge difference. This screams: Statistics problem

Fix

Run: ANALYZE users; or VACUUM ANALYZE users;

Then: EXPLAIN ANALYZE again.

Often the plan changes immediately.

3.3 Function on the column: index on raw value is not enough

```
CREATE INDEX idx_users_email ON users(email);
```

```
EXPLAIN ANALYZE
SELECT *
FROM users
WHERE LOWER(email) = 'john@test.com';
```

```
Seq Scan on users (cost=0.00..22334.00 rows=1 width=64)
(actual time=0.033..145.127 rows=1 loops=1)
  Filter: (lower(email) = 'john@test.com')
  Rows Removed by Filter: 999999
Execution Time: 145.231 ms
```

Fix:

```
CREATE INDEX idx_users_email_lower ON users (lower(email));
```

```
EXPLAIN ANALYZE
SELECT *
FROM users
WHERE LOWER(email) = 'john@test.com';
```

```
Index Scan using idx_users_email_lower on users (cost=0.42..8.44 rows=1 width=64)
(actual time=0.032..0.034 rows=1 loops=1)
Index Cond: (lower(email) = 'john@test.com')
Execution Time: 0.051 ms
```

3.4 Leading wildcard search: B-tree cannot jump into the middle of a string

```
CREATE INDEX idx_users_name ON users(name);
```

```
EXPLAIN ANALYZE
SELECT *
FROM users
WHERE name LIKE '%john%';
```

```
Seq Scan on users (cost=0.00..21334.00 rows=1000 width=32)
(actual time=0.020..138.540 rows=1042 loops=1)
Filter: (name ~~ '%john%' :: text)
Rows Removed by Filter: 998958
Execution Time: 139.113 ms
```

Fix for substring search:

```
CREATE EXTENSION pg_trgm;
CREATE INDEX idx_users_name_trgm ON users USING gin (name gin_trgm_ops);
```

```
EXPLAIN ANALYZE
SELECT *
FROM users
WHERE name LIKE '%john%';
```

```
Bitmap Heap Scan on users (cost=35.00..410.00 rows=1000 width=32)
(actual time=1.211..4.842 rows=1042 loops=1)
Recheck Cond: (name ~~ '%john%' :: text)
Execution Time: 5.201 ms
```

3.5 Composite index, wrong order

```
CREATE INDEX idx_orders_user_date ON orders(user_id, created_at);
```

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE created_at >= NOW() - INTERVAL '1 day';
```

```
Seq Scan on orders (cost=0.00..40234.00 rows=12000 width=48)
(actual time=0.091..186.445 rows=11912 loops=1)
Filter: (created_at >= (now() - '1 day'))
Execution Time: 188.921 ms
```

Correct usage when the left-most prefix is present:

```
EXPLAIN ANALYZE
SELECT *
FROM orders
WHERE user_id = 101
AND created_at >= NOW() - INTERVAL '1 day';
```

```
Index Scan using idx_orders_user_date on orders (cost=0.42..51.33 rows=31 width=48)
(actual time=0.041..0.198 rows=28 loops=1)
Index Cond: (user_id = 101 AND created_at >= (now() - '1 day'))
Execution Time: 0.223 ms
```

4. The offset problem: why pagination gets expensive

OFFSET is one of the most common index traps in production systems. It looks harmless in code, but it forces PostgreSQL to walk past the skipped rows before it can return the next page.

For small offsets, the cost is hidden. For deep pages, the database still has to find and discard the first N rows. That work grows linearly with OFFSET.

4.1 Bad pattern: OFFSET pagination

```
CREATE INDEX idx_transactions_created_at_id_desc
ON transactions (created_at DESC, id DESC);

EXPLAIN ANALYZE
SELECT id, created_at, amount, status
FROM transactions
ORDER BY created_at DESC, id DESC
LIMIT 20 OFFSET 100000;
```

Limit (cost=5210.42..5211.46 rows=20 width=48)
(actual time=71.144..71.152 rows=20 loops=1)
-> Index Scan using idx_transactions_created_at_id_desc on transactions
(cost=0.42..52100.42 rows=1000000 width=48)
(actual time=0.040..68.933 rows=100020 loops=1)
Planning Time: 0.182 ms
Execution Time: 71.192 ms

Notice that an index is still used, but PostgreSQL must read 100,020 rows to return just 20. Deep offsets are therefore an application-level scalability issue, not only an indexing issue.

4.2 When OFFSET gets even worse

If there is no suitable ordering index, PostgreSQL often falls back to a sort on top of a sequential scan.

```
EXPLAIN ANALYZE
SELECT id, created_at, amount, status
FROM transactions
ORDER BY created_at DESC
LIMIT 20 OFFSET 100000;
```

Limit (cost=64100.00..64100.05 rows=20 width=48)
(actual time=312.440..312.447 rows=20 loops=1)
-> Sort (cost=61500.00..64000.00 rows=1000000 width=48)
(actual time=280.125..305.330 rows=100020 loops=1)
Sort Key: created_at DESC
Sort Method: external merge Disk: 38400kB
-> Seq Scan on transactions (cost=0.00..18334.00 rows=1000000 width=48)
(actual time=0.018..84.771 rows=1000000 loops=1)
Execution Time: 313.011 ms

4.3 Best fix: keyset pagination

Keyset pagination replaces OFFSET with a cursor predicate. Instead of asking PostgreSQL to skip 100,000 rows, you ask it to continue from the last row of the previous page.

```
-- first page
SELECT id, created_at, amount, status
FROM transactions
ORDER BY created_at DESC, id DESC
LIMIT 20;

-- next page, using the last row from the previous page as the cursor
SELECT id, created_at, amount, status
FROM transactions
WHERE (created_at, id) < (:last_created_at, :last_id)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

Why this is better:

- The predicate is sargable and matches the index order.
- The database can stop immediately after reading the next 20 rows.
- The query latency stays stable even on deep pages.

-

- It avoids duplicates and missing rows when paired with a deterministic order, such as created_at plus id.

```
EXPLAIN ANALYZE
SELECT id, created_at, amount, status
FROM transactions
WHERE (created_at, id) < ('2026-06-14 10:30:00+00', 987654321)
ORDER BY created_at DESC, id DESC
LIMIT 20;

Limit (cost=0.42..2.31 rows=20 width=48)
(actual time=0.043..0.081 rows=20 loops=1)
-> Index Scan using idx_transactions_created_at_id_desc on transactions
(cost=0.42..9800.42 rows=103421 width=48)
(actual time=0.041..0.074 rows=20 loops=1)
Index Cond: (ROW(created_at, id) < ROW('2026-06-14 10:30:00+00'::timestampz, 987654321))
Execution Time: 0.096 ms
```

4.4 Other pagination fixes

- Use keyset pagination for user-facing lists, activity feeds, transactions, audit logs, and merchant events.
- If total page counts are required, cache counts asynchronously rather than forcing deep OFFSET pagination on every request.
- For export jobs, use streaming or cursor-based reads instead of OFFSET.
- If you must keep page numbers for the UI, translate them internally to a cursor-based API where possible.
- For very large analytical reads, consider pre-aggregated tables or read replicas instead of repeatedly scanning the hot OLTP table.

4.5 Why != and <> often ignore indexes

Inequality predicates such as != and <> are often misunderstood. Even when a column is indexed, PostgreSQL may still prefer a Sequential Scan if the predicate matches a large share of the table.

That decision is usually correct. Indexes are strongest when they can narrow the result set quickly. If the planner expects to return most of the table anyway, a sequential read is often cheaper than random index lookups plus heap fetches.

Example:

```
CREATE INDEX idx_users_status ON users(status);

SELECT * FROM users WHERE status <> 'ACTIVE';
```

If 95% of rows are non-ACTIVE, PostgreSQL will usually choose a Seq Scan because the predicate is not selective enough.

In a payments system, the same pattern appears in operational queries such as status != 'SUCCESS' when investigating failures. If only a small fraction of payments are unsuccessful, the predicate can still be index-friendly. If most rows match, the index will not help much.

A better query design is often to use a positive predicate instead of a negative one:

```
SELECT * FROM transactions WHERE status IN ('FAILED', 'TIMEOUT');
```

That shape is usually easier for PostgreSQL to estimate and often maps more cleanly to index usage.

When the business case is focused on the non-success subset, a partial index is usually the strongest solution:

```
CREATE INDEX idx_failed_transactions ON transactions(created_at) WHERE status <> 'SUCCESS';
```

Takeaway: PostgreSQL indexes are optimized for finding what you want, not for excluding what you do not want. Predicates such as =, IN, BETWEEN, >, and < are generally more index-friendly than !=, <>, NOT IN, and some NOT EXISTS patterns.

5. When PostgreSQL can use an index, but still chooses Seq Scan

This is the other half of the problem. PostgreSQL may technically be able to use an index, but still decide that the sequential plan is cheaper.

5.1 Small tables

For tiny tables, reading the whole table sequentially can be faster than doing random index lookups.

5.2 Large result sets

If a predicate returns a substantial fraction of the rows, Seq Scan is often the correct choice.

5.3 Stale statistics

```
ANALYZE transactions;  
VACUUM ANALYZE transactions;
```

After significant data churn, stale stats can make the planner underestimate or overestimate row counts. That affects join order, scan type, and index choice.

5.4 Cost constants

```
SHOW random_page_cost;  
SHOW seq_page_cost;
```

On SSD-backed systems, the default random_page_cost may be conservative. Lowering it can make index usage more likely, but only after careful testing.

6. Practical decision matrix

Symptom	Typical plan	Root cause	Preferred fix
LOW selectivity	Seq Scan	Too many rows qualify	Keep Seq Scan; reconsider the index only if distribution changes
Function on column	Seq Scan	Predicate does not match index expression	Add an expression index or rewrite the query
Datatype mismatch	Seq Scan	Implicit cast changes the operator	Align types; cast to the column type
Leading wildcard	Seq Scan / Bitmap	B-tree cannot search prefix-less patterns	Use pg_trgm or change the search pattern
Deep OFFSET	Index Scan + many discarded rows	Pagination walks through skipped rows	Use keyset pagination
Stale statistics	Wrong scan choice	Planner estimates are inaccurate	ANALYZE and tune autovacuum/statistics

PostgreSQL is not ignoring your index randomly.
It is either unable to use it, or it is making a cost-based choice that is probably correct for the data it sees.
The job of the engineer is to make the query, statistics, and index definition line up with the real access pattern.

References & Further Reading

1. pgMustard — Why Isn't Postgres Using My Index?

The article that inspired part of this meetup.

Topics covered:

- PostgreSQL planner decision-making
- Why indexes are sometimes ignored
- Functions preventing index usage
- Datatype mismatches
- Statistics and cardinality estimation
- Cost-based optimization
- Sequential Scan vs Index Scan tradeoffs

2. PostgreSQL Using EXPLAIN Documentation

The most important official resource for understanding query plans.

Topics covered:

- EXPLAIN
- EXPLAIN ANALYZE
- Cost estimation
- Actual vs estimated rows

- Understanding execution plans

Highly recommended for every backend engineer working with PostgreSQL.

3. PostgreSQL Planner Statistics Documentation

Essential reading for understanding:

- Cardinality estimation
- Histograms
- Most Common Values (MCV)
- ANALYZE
- `default_statistics_target`
- Why PostgreSQL chooses Seq Scan instead of Index Scan

This is arguably the most important topic after learning EXPLAIN.

4. pganalyze — Basics of PostgreSQL Query Planning

Excellent practical explanation of:

- Query planning
- Plan trees
- Cost-based optimization
- Planner behavior

A great bridge between official documentation and production engineering.

5. Use The Index, Luke!

One of the best database performance books/websites ever written.

Especially useful for:

- Pagination
- OFFSET performance issues
- Keyset pagination
- Composite indexes
- Covering indexes

Community members consistently recommend it as a foundational indexing resource.
